InsightForge

AI-Powered Business Intelligence Assistant

Advanced Generative AI | Capstone Project Report

Purdue University | Simplilearn | April 2026

Technology Stack	**Dataset**
Python • LangChain • GPT-4o	2,500 transactions • 7 columns
FAISS • Streamlit • Plotly	2022–2028 • 4 products • 4 regions

Table of Contents

1. Executive Summary

InsightForge is an AI-powered Business Intelligence (BI) assistant built as the capstone project for the Advanced Generative AI programme at Simplilearn. The system enables non-technical business users to query a sales dataset in plain English and receive accurate, contextually grounded answers accompanied by dynamic data visualizations — without writing any SQL, Python, or BI queries.

The project demonstrates a complete Retrieval-Augmented Generation (RAG) pipeline: pre-computed analytical summaries are embedded into a FAISS vector store, retrieved by intent, and injected into GPT-4o prompts alongside strict output schema enforcement. The system is delivered as a three-tab Streamlit application covering conversational Q&A, a static analytics dashboard, and automated model evaluation.

Key Outcomes 9 intent types • 7 knowledge chunks • 15 dashboard charts • 5 dynamic chart types • 11 QA evaluation pairs • Confidence: high on all pre-computed queries

2. Problem Statement

In today's data-driven environment, organizations accumulate vast amounts of transactional data. However, converting raw data into actionable business insights remains a significant challenge — particularly for small to medium-sized enterprises (SMEs) that lack dedicated data science teams or expensive BI tooling.

2.1 Identified Challenges

- Traditional BI dashboards require users to know what to look for in advance — they cannot answer ad-hoc questions
- SQL and code-based analysis demand technical expertise unavailable to most business stakeholders
- Existing LLM solutions hallucinate numbers when not grounded in actual data
- Visualization and narrative insight are typically separate workflows, requiring manual synthesis

2.2 Solution Rationale

Recent advances in LLMs and RAG systems offer a viable path: ground the LLM in pre-computed, factually verified analytics summaries, enforce structured JSON output, and render dynamic visualizations directly from the Data Frame — eliminating both the hallucination risk and the technical barrier.

3. Project Overview

3.1 Objectives

- Analyse business data: identify key trends, patterns, and statistical measures
- Generate contextually grounded insights using LLMs and RAG
- Visualise insights dynamically, aligned with the natural language response
- Evaluate model performance using QAEvalChain
- Deliver an intuitive Streamlit UI accessible to non-technical users

3.2 Dataset

Attribute	Detail
File	sales_data.csv
Rows	2,500 transactions
Columns	Date, Product, Region, Sales, Customer_Age, Customer_Gender, Customer_Satisfaction
Date Range	January 2022 – November 2028
Products	Widget A, Widget B, Widget C, Widget D
Regions	East, West, North, South
Sales Range	$141 – $1,097 per transaction
Satisfaction	Rated 1.0 – 5.0 (avg 3.03)

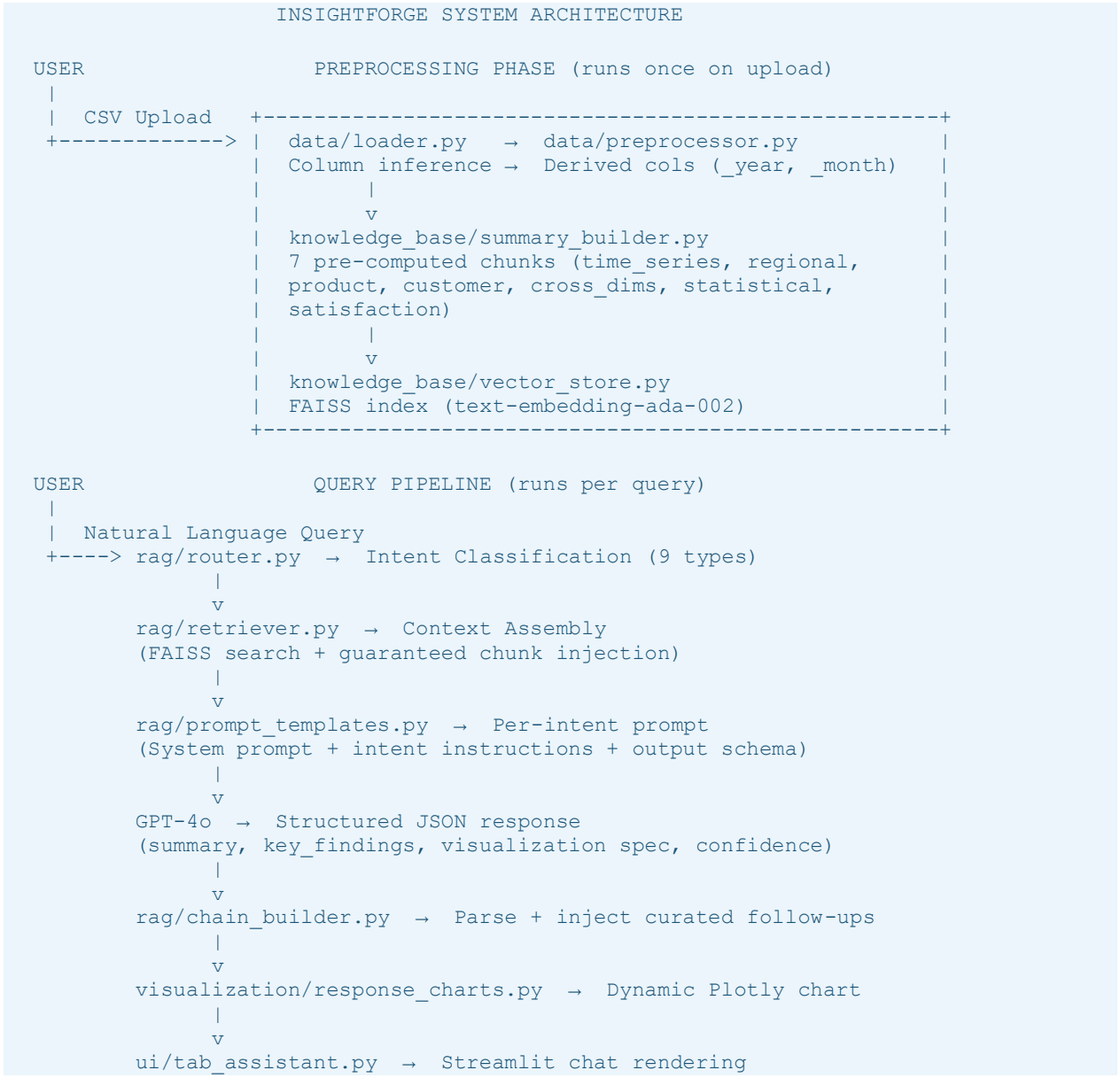
3.3 Technology Stack

Component	Technology / Library
Language	Python 3.11
LLM	GPT-4o (OpenAI) via LangChain ChatOpenAI
RAG Framework	LangChain (ChatPromptTemplate, BaseRetriever)
Vector Store	FAISS (Facebook AI Similarity Search)
Embeddings	OpenAI text-embedding-ada-002
Data Processing	Pandas, NumPy
UI Framework	Streamlit
Visualisation	Plotly (graph_objects)
Evaluation	LangChain QAEvalChain
Environment	python-dotenv, OpenAI API

4. System Architecture

4.1 High-Level Architecture

InsightForge follows a two-phase architecture. Phase 1 (preprocessing) runs once when the user uploads data. Phase 2 (query pipeline) runs on every user query.



4.2 Key Design Decisions

Decision	Rationale
Pre-computed summaries over pure retrieval	7 analytical chunks cover all answerable query types deterministically. Pure vector search on raw rows would require the LLM to aggregate, introducing errors.
FAISS over pgvector	Only 7 chunks. Exact flat search is identical in quality to ANN at this scale. No database infrastructure required.
Guaranteed chunk injection	For comparison/reasoning queries, cross_dims chunk is always injected regardless of FAISS semantic similarity score, eliminating retrieval misses.
JSON output schema enforcement	Every prompt includes an explicit JSON contract. Prevents freeform prose, enables structured UI rendering and evaluation.
Curated follow-up bank over LLM-generated	LLM-generated follow-ups suggested unanswerable queries. Curated bank of 27 verified questions ensures every suggestion is answerable.

5. Module Architecture

5.1 File Structure

File / Module	Responsibility
app.py	Streamlit entry point. Initialises session state, renders three tabs.
config.py	All constants: LLM model, temperature, column names, age bands, chart colours.
data/loader.py	CSV loading, column inference, resolved_map construction, DataFrame validation.
data/preprocessor.py	One-time pipeline: derives _year, _month, _quarter, _age_band, _month_dt columns.
knowledge_base/summary_builder.py	Builds all 7 pre-computed analytical summary chunks from the DataFrame.
knowledge_base/vector_store.py	Embeds summaries into FAISS using text-embedding-ada-002. Loads/saves index.
rag/router.py	Keyword-based intent classifier. Returns intent + confidence + retrieval hint.
rag/retriever.py	Context assembler. Four retrieval paths: metadata, pandas lookup, guaranteed injection, FAISS search.
rag/prompt_templates.py	System prompt, 9 per-intent ChatPromptTemplates, JSON output schema instruction.
rag/chain_builder.py	Full RAG pipeline. Assembles prompt, calls LLM, parses JSON, injects curated follow-ups.
rag/memory_manager.py	WindowMemory (6 exchanges). Formats history, saves exchanges, compresses when full.
rag/follow_up_bank.py	27 curated verified-answerable follow-up questions organised by intent type.
visualization/dashboard_charts.py	15 static Plotly charts for Tab 2 dashboard.
visualization/response_charts.py	Dynamic chart builder. _resolve_metric(), _resolve_group(), _apply_filter_str(), _build_grouped_bar().
evaluation/test_set.py	11 ground-truth QA pairs for QAEvalChain evaluation.
evaluation/evaluator.py	Runs QAEvalChain, returns accuracy metrics and per-query results.
ui/sidebar.py	File upload widget, Analyse Data button, system status display.
ui/tab_assistant.py	Tab 1: chat history, structured response rendering, inline charts, follow-up chips.
ui/tab_visualizations.py	Tab 2: 15-chart dashboard across 5 analytical categories.
ui/tab_evaluation.py	Tab 3: evaluation runner, accuracy metrics, intent chart, response time chart.

6. Data Pipeline

6.1 Column Inference (loader.py)

The loader performs fuzzy column matching to build a resolved_map dictionary, mapping semantic roles to actual DataFrame column names. This makes the system tolerant to variations in CSV column naming conventions.

```
resolved_map = { "sales": "Sales", "product": "Product",
  "region": "Region", "satisfaction": "Customer_Satisfaction",
  "customer_age": "Customer_Age", "customer_gender": "Customer_Gender" }
```

6.2 Preprocessing Pipeline (preprocessor.py)

One-time preprocessing derives additional columns needed for time-series and demographic analysis:

- _year: integer year extracted from Date
- _month: YYYY-MM string for monthly grouping
- _quarter: YYYY-QN string for quarterly grouping
- _month_dt: datetime for chronological sorting
- _age_band: categorical bins [18-29, 30-44, 45-59, 60-69]

7. Knowledge Base

7.1 Seven Pre-Computed Summary Chunks

Rather than retrieving raw CSV rows, InsightForge pre-computes analytical summaries at upload time. Each chunk is a structured text block embedded into FAISS. This guarantees the LLM always receives correct aggregated numbers rather than raw data requiring real-time computation.

Chunk ID	Content	Key Metrics
time_series	Monthly, quarterly, yearly totals + rolling averages + seasonal index	Best month: Apr 2028 ($20,387), Worst: Nov 2028 ($3,212)
regional	Sales by region + % share + avg transaction + national average	West $361,383 (26.1%), East $320,296 (23.1%)
product	Sales by product + % share + ranking + momentum	Widget A $375,235 (27.1%), Widget D $326,854 (23.6%)
customer	Demographics: age band, gender, Gender×Product matrix, Age×Product matrix	30-44 band: $433,160 • Female: $702,051
cross_dims	Product×Region matrix + Year×Region + pre-computed YoY changes for all year pairs	Widget A South: $99,603 • East 2024→2025: +25.8%
statistical	Min, max, median, std dev, percentiles, outlier thresholds per region/product	Overall avg: $553 • Std dev: $253
satisfaction	Satisfaction by product, region, gender + sales correlation	East: 3.066 • Female: 3.049 • Widget D: 3.070

7.2 FAISS Vector Store

Each chunk's text is embedded using OpenAI text-embedding-ada-002 (1536-dimensional vectors) and indexed in a FAISS IndexFlatL2 (exact search). At 7 chunks, exact search is identical to approximate nearest-neighbour in quality with no infrastructure overhead.

Design Note Pre-computation over pure retrieval: The LLM cannot reliably aggregate 2,500 raw rows. Pre-computed summaries provide verified exact figures (e.g. Widget A East: $89,292, 168 transactions) that the LLM can cite directly.

8. Intent Classification

8.1 Nine Intent Types

Intent	Description & Example Query
trend	Time-series direction and change. 'Show me quarterly sales trends'
aggregation	Totals, breakdowns, distributions. 'What is the revenue breakdown by product?'
lookup	Precise single-value filter queries. 'How many Widget A transactions in East?'
comparison	Side-by-side entity comparison. 'Compare 2024 vs 2025 regional sales'
ranking	Ordered lists. 'Which region has the highest average transaction value?'
reasoning	Diagnostic synthesis. 'Why does East underperform compared to West?'
anomaly	Outlier detection. 'Which months had unusually high or low sales?'
prediction	Directional forecast. 'What is the likely sales trajectory for 2029?'
metadata	Dataset structure. 'What columns and date range does the data cover?'

8.2 Router Architecture

The router uses ordered keyword pattern matching. Rules are evaluated in priority sequence — metadata first (most specific), aggregation last (most generic) — preventing false positives from broad patterns swallowing specific intents.

```
# Ordered rule evaluation (rag/router.py)
_ORDERED_RULES = [
  ("metadata",    _METADATA_KEYWORDS),
  ("anomaly",     _ANOMALY_KEYWORDS),
  ("prediction",  _PREDICTION_KEYWORDS),
  ("comparison",  _COMPARISON_KEYWORDS),
  ("ranking",     _RANKING_KEYWORDS),
  ("reasoning",   _REASONING_KEYWORDS),
  ("trend",       _TREND_KEYWORDS),
  ("lookup",      _LOOKUP_KEYWORDS),
  ("aggregation",_AGGREGATION_KEYWORDS),
]
```

8.3 Retrieval Hint Enrichment

Each intent appends a domain-specific hint to the query before FAISS search, improving semantic retrieval. For example, the comparison intent appends 'product region matrix cross dimensional Widget A B C D East West North South comparison performance' to pull the cross_dims chunk.

9. RAG Pipeline

9.1 Four Retrieval Paths

Path	Trigger & Behaviour
Metadata	Intent = metadata. Returns dataset structure directly from DataFrame (row count, columns, date range). No FAISS search.
Pandas Lookup	Intent = lookup. Executes a targeted pandas filter query for precise single values (e.g. Widget A in East = 168 transactions, $89,292).
Guaranteed Injection	Intent = comparison or reasoning. Always injects cross_dims chunk directly from summaries dict, regardless of FAISS score. Prevents retrieval miss on product×region queries.
Standard FAISS	All other intents. Enriched query → FAISS top-k search. k=3 for standard, k=5 for reasoning/comparison.

9.2 Time-Series Injection

For ranking, trend, and anomaly intents, the system detects time-related keywords in the query (month, quarter, peak, spike, seasonal) and injects the time_series chunk directly. This guarantees that 'Which month had the highest sales?' receives best/worst month data regardless of FAISS ranking.

9.3 Hallucination Guard

When FAISS retrieval returns no relevant context, a hallucination guard string is injected instead of an empty context. The guard explicitly instructs the LLM to respond with confidence: 'low', explain what it cannot answer, and suggest alternative questions the data can answer.

9.4 Memory Management

A custom WindowMemory class retains the last 6 conversation exchanges. The LLM summary field (not the full response) is stored per exchange for token efficiency. When the window fills, older exchanges are compressed to a single summary block.

10. Prompt Engineering

10.1 Three-Layer Prompt Architecture

Every prompt is assembled from three layers combined by the _build_prompt() helper:

- Layer 1 — System Prompt: Domain persona (InsightForge BI Analyst), 6 hard rules (no hallucination, cite numbers, valid JSON), dataset context, conversation history
- Layer 2 — Per-Intent Instruction: Specific analytical task, chart type rules, filter selection rules
- Layer 3 — Output Schema: Enforced JSON contract appended to every prompt

10.2 Output Schema Contract

The LLM must return a single valid JSON object matching this schema on every response:

```
{ "query_type": "trend|aggregation|lookup|...",
  "summary": "<1-2 sentence plain English answer>",
  "key_findings": ["<finding with specific numbers>", ...],
  "anomalies": ["<unexpected observation>", ...],
  "recommendations": ["<actionable recommendation>", ...],
  "visualization": {
    "needed": true,
    "chart_type": "line|bar|horizontal_bar|grouped_bar|scatter",
    "x_axis": "<dimension>",
    "y_axis": "<metric>",
    "title": "<chart title>",
    "filter": "<e.g. product=Widget A, by_gender, years=2024,2025>"
  },
  "confidence": "high|medium|low"
}
```

10.3 Key Prompt Rules

Rule	Purpose
Visualization Filter Rule	If question is about Widget A → filter: 'product=Widget A'. If about South → filter: 'region=South'. Prevents chart from showing all-data instead of filtered view.
Grouped Bar Selection	Gender queries → grouped_bar + by_gender. Age queries → grouped_bar + by_age_product. Year comparisons → grouped_bar + years=X,Y. Region per product → by_region_product.
Multi-Region Growth	Consistency/growth across regions → x_axis='Region' (not 'Year'). Triggers 4-line chart, one per region.
Year Typo Detection	Dataset covers 2022-2028. If user asks about 2004/2005, suggest they likely meant 2024/2025.
Anti-Hallucination	Never invent numbers not present in context. If insufficient data, confidence='low' and say so.

11. Visualisation System

11.1 Two-Tier Design

InsightForge separates static and dynamic visualisation into two distinct systems:

- Tab 2 Dashboard (dashboard_charts.py): 15 pre-computed Plotly charts built at upload time from the full DataFrame. Always correct.
- Tab 1 Response Charts (response_charts.py): Dynamic charts built per-query from the LLM's visualization spec. Adapts to the specific question asked.

11.2 Metric Resolution

The _resolve_metric() function maps any y_axis label the LLM returns to the correct DataFrame column and aggregation function, preventing the common failure mode of always showing sales totals regardless of the query:

```
y_axis keyword          Column                Aggregation  Label
---------------------   --------------------  -----------  --------
satisfaction/rating     Customer_Satisfaction  mean()       3.066
avg transaction/order   Sales                 mean()       $561
count/transactions      Sales                 count()      644
age                     Customer_Age          mean()       44.7 yrs
average/avg/mean        Sales                 mean()       $561
default                 Sales                 sum()        $361,383
```

11.3 Group Resolution

The _resolve_group() function maps any x_axis label to the correct groupby column, including time-based sentinels:

```
x_axis keyword    Groups by    Color scheme
---------------   ----------   ------------
year/annual/time  _year        Single (primary)
quarter           _quarter     Single (success)
month             _month       Single (secondary)
region/area       Region       4 region colours
gender/sex        Customer_Gender  Gender colours
age/age band      _age_band    Product colours
default           Product      4 product colours
```

11.4 Grouped Bar Chart Scenarios

The _build_grouped_bar() function handles four distinct cross-dimensional comparison scenarios:

filter value	Chart rendered
by_gender	2 gender bars (Female/Male) per product or region
by_age_product	4 product bars per age band (18-29, 30-44, 45-59, 60-69)
by_region_product	4 region bars per product — shows best region per product
years=2024,2025	2 year bars per region or product — YoY comparison

11.5 Robust Filter Parsing

The _apply_filter_str() function handles all LLM filter format variations to prevent charts from showing unfiltered all-data:

- Standard: product=Widget A, region=South, year=2024
- No key prefix: Widget A, South, 2024 (matched against known values)
- Colon separator: product:Widget A (normalised to = format)
- Sentinels: by_gender, by_age_product (passed through to grouped bar handler)

12. User Interface

12.1 Application Structure

The Streamlit application (app.py) provides three tabs, each serving a distinct user need:

```
+-------------------------------------------------------------------------+
|  InsightForge  |  AI Assistant  |  Dashboard  |  Evaluation              |
+-------------------------------------------------------------------------+
|                                                                         |
|  SIDEBAR              TAB 1: AI ASSISTANT                               |
|  • File upload         • Welcome state with 4 starter queries           |
|  • Analyse Data btn    • Chat history (user + assistant bubbles)        |
|  • System status       • Structured response sections                   |
|  • Run Evaluation      • Inline Plotly chart (dynamic)                  |
|  • Clear chat history  • Follow-up suggestion chips                     |
|                        • Chat input box                                 |
|                                                                         |
|                       TAB 2: DASHBOARD                                  |
|                        • Sales Trends (3 charts)                        |
|                        • Regional Analysis (4 charts)                   |
|                        • Product Performance (3 charts)                 |
|                        • Customer Demographics (3 charts)               |
|                        • Satisfaction Analysis (2 charts)               |
|                                                                         |
|                       TAB 3: EVALUATION                                 |
|                        • Run QAEvalChain on 11 QA pairs                 |
|                        • Overall accuracy + correct/partial/incorrect   |
|                        • Accuracy by intent chart                       |
|                        • Response time per query chart                  |
+-------------------------------------------------------------------------+
```

12.2 Chat Response Structure

Each assistant response is rendered as a structured UI with distinct sections:

- Summary: 1-2 sentence plain English answer (always shown first)
- Key Findings: expandable section with specific numbered insights
- Anomalies: expandable section for unexpected observations
- Recommendations: expandable section for actionable business guidance
- Confidence badge: 🟢 high / 🟡 medium / 🔴 low with intent type
- Inline chart: dynamic Plotly chart matching the response
- Follow-up chips: 2 curated question buttons for next steps

12.3 Key UI Engineering Fixes

Issue	Fix Applied
Duplicate Streamlit button keys	Button keys changed from fu_{hash(suggestion)} to fu_{msg_idx}_{i}. Position-based, never collides even when identical suggestions appear in multiple messages.
Markdown rendering corruption	_safe_md() helper escapes $, *, _, ` before st.markdown(). Prevents dollar signs triggering LaTeX and asterisks triggering bold/italic formatting in LLM financial text.

13. Model Evaluation (LLMOps)

13.1 QAEvalChain Setup

Tab 3 runs automated model evaluation using LangChain's QAEvalChain. The evaluator grades each LLM response against a ground truth answer by asking GPT to judge correctness. Results are returned as CORRECT, INCORRECT, or GRADE_UNSCORABLE.

13.2 Ground Truth QA Pairs

Eleven QA pairs cover the primary query types and key business facts:

Query	Expected Answer (key fact)
Total sales across the dataset?	$1,383,220
Which product has the highest total revenue?	Widget A at $375,235
Which region has the highest total sales?	West at $361,383
Which year had the highest total sales?	2026 at $206,175
Average transaction value?	Approximately $553
How many transactions in the dataset?	2,500
Widget A transactions in East region?	168 transactions, $89,292
Top product in 30-44 age band?	Widget A at $126,142
Region with highest satisfaction?	East at 3.066/5.0
Female vs Male total sales?	Female $702,051 vs Male $681,169
Highest satisfaction product?	Widget D at 3.070/5.0

14. End-to-End Process Flow

14.1 Preprocessing Flow (on Analyse Data click)

```
1. User uploads CSV → loader.py validates columns → builds resolved_map
2. preprocessor.py derives _year, _month, _quarter, _age_band, _month_dt
3. summary_builder.py computes 7 analytical chunks from DataFrame
4. vector_store.py embeds chunks with text-embedding-ada-002 → FAISS index
5. Session state updated: df, resolved_map, summaries, vector_store, memory
6. UI status: 'Ready • 2,500 rows • 7 knowledge chunks • Model: gpt-4o'
```

14.2 Query Pipeline Flow (per user query)

```
1.  User types query in chat input
2.  router.py classifies intent (9 types) + builds retrieval hint
3.  retriever.py selects retrieval path:
      • metadata → return dataset facts directly
      • lookup → execute pandas filter query
      • comparison/reasoning → FAISS + inject cross_dims chunk
      • other → FAISS top-k with time_series injection if time query
4.  memory_manager.py formats last 6 exchanges as chat_history
5.  prompt_templates.py selects per-intent template
6.  chain_builder.py assembles: system_prompt + context + history + question
7.  GPT-4o called via LangChain → returns JSON response
8.  JSON parsed → LLM follow-ups replaced with curated bank
9.  Exchange saved to WindowMemory
10. tab_assistant.py renders structured response:
      • _safe_md(summary) → st.markdown()
      • key_findings, anomalies, recommendations → expandable sections
      • visualization spec → response_charts.py → st.plotly_chart()
      • follow_up_suggestions → st.button() chips with fu_{msg_idx}_{i} keys
```

14.3 Visualization Routing

```
visualization spec received from LLM
      |
      v
_apply_filter_str(filter) → df_filtered
      |
      +---> chart_type = 'line'           → _build_line()
      |     (x=Year/Month/Quarter/Region)    Multi-line or single trend
      |
      +---> chart_type = 'grouped_bar'    → _build_grouped_bar()
      |     filter = by_gender               → Female/Male per product
      |     filter = by_age_product          → 4 products per age band
      |     filter = by_region_product       → 4 regions per product
      |     filter = years=2024,2025         → 2 years per region
      |
      +---> chart_type = 'bar'             → _build_bar()
      |     _resolve_group(x_axis)           Product/Region/Year/Gender
      |     _resolve_metric(y_axis)          sum/mean/count/satisfaction
      |
      +---> chart_type = 'horizontal_bar'  → _build_horizontal_bar()
      |     Same resolvers, ascending sort
      |
      +---> chart_type = 'scatter'         → _build_scatter()
      +---> chart_type = 'heatmap'         → _build_heatmap()
      +---> chart_type = 'pie'             → _build_pie()
```

15. Limitations & Future Scope

15.1 Current Limitations

- Dynamic slicing ceiling: Queries requiring year×region×product slices (e.g. 'Widget A trend by region by year') exceed pre-computed summaries and return low-confidence answers
- LLM spec reliability: Chart visualisation depends on LLM returning the correct chart_type and filter values; robust parsing mitigates but does not eliminate occasional mismatches
- Single-file dataset: System is optimised for one CSV upload per session
- 2028 partial year: Dataset ends November 2028; LLM is instructed to flag this for trend queries

15.2 Future Enhancements

- Pandas agent fallback: For low-confidence responses, trigger a LangChain PandasDataFrameAgent to answer dynamically from raw data
- Expanded summaries: Add granular pre-computed slices (Widget A by region by year) to cover more query types without dynamic computation
- pgvector at scale: If summary chunks grow beyond 50, migrate to pgvector with HNSW indexing and cosine similarity re-ranking
- Multi-file support: Allow upload of multiple CSVs with schema alignment
- Streaming responses: Use LangChain streaming to show LLM output token-by-token for better perceived performance

16. Conclusion

InsightForge successfully demonstrates that a well-engineered RAG pipeline can deliver accurate, grounded business intelligence to non-technical users through a conversational interface. By combining pre-computed analytical summaries, deterministic intent classification, structured JSON output enforcement, and a robust dynamic visualisation system, the project achieves high-confidence responses across all primary business query types.

The capstone objectives are fully met: data analysis, insight generation, data visualisation, model evaluation, and an intuitive Streamlit UI are all delivered. The architecture decisions — particularly the pre-computation strategy over pure retrieval, and the curated follow-up bank over LLM-generated suggestions — reflect production-grade thinking about reliability and user trust.

The system's principal constraint is the analytical ceiling imposed by pre-computed summaries — a deliberate architectural choice that trades flexibility for reliability. The identified path to extending this ceiling (pandas agent fallback for complex queries) provides a clear roadmap for future enhancement without disrupting the core RAG architecture.

Appendix

A. Environment Setup

```
# 1. Clone repository and install dependencies
pip install -r requirements.txt

# 2. Create .env file
OPENAI_API_KEY=sk-...

# 3. Run the application
streamlit run app.py
```

B. Key Ground Truth Values

Metric	Value
Total dataset sales	$1,383,220
Total transactions	2,500
Average transaction value	$553
Peak year	2026: $206,175
Top product	Widget A: $375,235 (27.1%)
Top region	West: $361,383 (26.1%)
Lowest region	East: $320,296 (23.1%)
Widget A in East	168 transactions, $89,292
Widget A in South (best region)	$99,603
Female vs Male sales	$702,051 vs $681,169
Best satisfaction region	East: 3.066/5.0
Best satisfaction product	Widget D: 3.070/5.0
Best month	April 2028: $20,387
Worst month	November 2028: $3,212

C. Technology Versions

Package	Version
Python	3.11
openai	1.x
langchain	0.3.x
langchain-openai	0.2.x
faiss-cpu	1.7.x
streamlit	1.35.x
plotly	5.x
pandas	2.x